
Question 1

```
gcl<-function(a,b,m){  
  x<-(a*x+b)%m  
  
  return(x)  
}  
  
m=1171*8*7*9  
x<-789547  
b=11*13*17*19  
a=1171*4*7*3+1  
Seq1<-replicate(100,gcl(a,b,m))
```

Dans la première question, nous avons généré une séquence de 100 nombres pseudo-aléatoires en utilisant comme point de départ la graine $x_0 = 789547$ et en utilisant un générateur de nombres congruents linéaires (GCL) avec les paramètres suivants : $a = 1171 \times 4 \times 7 \times 3 + 1$, $b = 11 \times 13 \times 17 \times 19$, et $m = 1171 \times 8 \times 7 \times 9$.

Question 2

```
x<-25794  
Seq2<-replicate(100,gcl(a,b,m))
```

Dans la deuxième question, une autre séquence de 100 nombres pseudo-aléatoires a été générée en utilisant comme point de départ la graine $x_0 = 25794$ avec le même générateur de nombres congruents linéaires (GCL) que précédemment.

Question 3

```
MaxGeom<-dgeom(0,0.1)  
  
SeqUniFerme<-Seq1/(m-1)  
SeqMaxPi<-SeqUniFerme*MaxGeom
```

Dans la troisième question, pour trouver le maximum de la fonction de masse de la loi géométrique de paramètre 0.1, nous considérons que la distribution de la loi géométrique est décroissante.

Ainsi, nous choisissons la première probabilité dans le support $\mathbb{N} = \{0, 1, 2, 3, \dots\}$, qui est "0".

- *MaxGeom* : Cette étape consiste à utiliser une distribution géométrique avec la fonction `dgeom(0, 0.1)`, où 0.1 est le paramètre de probabilité de succès dans la distribution géométrique. Dans notre cas, *MaxGeom* correspond à la probabilité de succès pour le premier essai, soit $P(X = 0)$.
- *SeqUniFerme* : Nous avons normalisé la séquence "Seq1" en divisant chaque élément par $m - 1$, afin d'obtenir des valeurs dans l'intervalle $[0, 1]$ suivant la loi uniforme. Cela garantit que chaque élément de la séquence représente une probabilité uniforme dans cet intervalle.
- *SeqMaxPi* : En multipliant chaque élément de "SeqUniFerme" par *MaxGeom*, nous obtenons le maximum de la fonction de masse de la loi géométrique dans la séquence générée. Cette étape nous permet d'obtenir des valeurs dans l'intervalle $[0, \dots, \text{MaxGeom}]$, car nous n'avons pas besoin d'une probabilité supérieure à *MaxGeom*. Ainsi, *SeqMaxPi* représente la contribution de chaque élément de *SeqUniFerme* à la valeur maximale de la fonction de masse de la loi géométrique.

Question 4

```
PermierElement<-MaxGeom/(m-1)
i<-0

while(dgeom(i,0.1)>=PermierElement){

  i<-i+1
}
```

La quatrième question consiste à déterminer le premier entier i tel que la probabilité d'obtenir i succès dans une distribution géométrique de paramètre 0.1 soit supérieure ou égale à la valeur de `PermierElement`, qui est le premier élément de la séquence normalisée obtenue dans la troisième question.

- *PermierElement* : est défini comme le résultat de `MaxGeom` divisé par $m - 1$, où `MaxGeom` est la probabilité de la loi géométrique définie précédemment avec $dgeom(0, 0.1)$. Cela produit une valeur entre 0 et 1, représentant systématiquement la probabilité la plus petite de la séquence, en excluant 0.
- *le compteur i* : est initialisé à 0 et est utilisé pour parcourir les valeurs du support de la loi géométrique. En effet

$$\epsilon = \text{MaxGeom} / (m - 1)$$

représente la plus petite probabilité non nulle dans la séquence `SeqMaxPi`, car cette séquence est définie comme

$$\text{SeqMaxPi} = \left\{ 0, \frac{\text{MaxGeom}}{m-1}, \dots, (m-2) \frac{\text{MaxGeom}}{m-1}, \text{MaxGeom} \right\}$$

En appliquant la définition de la limite, nous cherchons i_0 tel que $\forall i > i_0, P_i < \epsilon$. Ainsi, notre support est défini dans $\{0, \dots, i_0\}$, car toutes les probabilités pour $i > i_0$ n'existent pas dans notre séquence `SeqMaxPi`.

Explication de la boucle

1. **Calcul de la probabilité géométrique** : À chaque itération de la boucle, la fonction `dgeom(i, 0.1)` est utilisée pour calculer la probabilité d'observer le premier succès (1) à l'essai i dans une distribution géométrique avec une probabilité de succès de 0.1. Cette probabilité diminue progressivement à mesure que i augmente, reflétant la décroissance caractéristique de la distribution géométrique.
2. **Comparaison avec `PermierElement`** : La condition `dgeom(i, 0.1) >= PermierElement` est évaluée à chaque itération pour déterminer si la probabilité de succès à l'essai i est encore suffisamment grande. Si cette condition est vraie, la boucle continue à incrémenter i , ce qui signifie que la probabilité d'observer un succès à l'essai i est supérieure ou égale à la valeur de `PermierElement`.
3. **Sortie de la boucle** : Lorsque la probabilité `dgeom(i, 0.1)` devient inférieure à `PermierElement`, la boucle s'arrête. À ce stade, i représente la plus petite valeur dans le support de la loi de Poisson où sa probabilité est inférieure à `PermierElement`. Cela signifie que i correspond au nombre d'essais nécessaires pour obtenir un succès avec une probabilité inférieure à `PermierElement`.

En résumé, cette boucle itère à travers les valeurs du support de la loi de Poisson, cherchant le premier i pour lequel la probabilité d'observer un succès devient inférieure à `PermierElement`, qui est dérivé de la distribution géométrique `MaxGeom` normalisée par $m - 1$.

Question 5

```
(SupportSize<-i+1)

## [1] 128

SeqUniOuvert<-Seq2/(m)

(SimulatedGeomSupp<-trunc(SeqUniOuvert*SupportSize))

## [1] 15 46 99 45 13 1 11 43 95 41 9 125 7 39 91 37 5 121
## [19] 3 35 87 33 1 118 0 31 84 30 125 114 124 27 80 26 121 110
## [37] 120 23 76 22 117 106 116 19 72 18 113 102 112 15 68 14 109 98
## [55] 108 11 64 10 105 94 104 8 60 6 102 90 100 4 56 2 98 86
## [73] 96 0 52 126 94 82 92 124 49 123 90 79 89 120 45 119 86 75
## [91] 85 116 41 115 82 71 81 112 37 111
```

- *SupportSize* : représente le cardinal du support de la loi géométrique. On ajoute 1 pour inclure 0 dans notre support $\{0, \dots, 127\}$, car le cardinal est égal à $127+1$.
- Remarque sur *SeqUniOuvert* : son intervalle est $[0, 1[$, le 1 est ouvert car on ne peut pas générer une probabilité égale à 1.
- *SimulatedGeomSupp* : ici, pour utiliser la méthode de rejet, nous avons effectué une multiplication élément par élément de *SeqUniOuvert* par le cardinal du support, qui est *SupportSize*, puis nous avons pris la partie entière du résultat avec la fonction "trunc".
- *trunc* : est une fonction mathématique qui retourne la partie entière d'un nombre réel.

Question 6

```
SimulatedGeomSeq<-c()
for (i in 1:100) {
  if(SeqMaxPi[i]<= dgeom(SimulatedGeomSupp[i],0.1)){
    SimulatedGeomSeq<-c(SimulatedGeomSeq,SimulatedGeomSupp[i])
  }
}
SimulatedGeomSeq

## [1] 13 1 9 5 3 1 0 18 14 10 6 0
```

Dans la sixième question, nous avons initialisé un vecteur appelé *SimulatedGeomSeq*. Ensuite, en connaissant le nombre d'itérations, nous avons utilisé une boucle *for* de 1 à 100. La condition de cette boucle était que si le *i*-ème élément de *SeqMaxPi* était inférieur ou égal à la fonction de masse du *i*-ème élément de *SimulatedGeomSupp*, alors nous avons affecté le *i*-ème élément de *SimulatedGeomSupp* au vecteur *SimulatedGeomSeq*.

Question 7

```
Xn<-Seq1[length(Seq1)]
Yn<-Seq2[length(Seq2)]
k<-100 #le 100 realisation du gcl
while(length(SimulatedGeomSeq)<100){
  x<-Xn
  Xn<-gcl(a,b,m)

  x<-Yn

  Yn<-gcl(a,b,m)

  Vi<-(Xn/(m-1))*MaxGeom
  Ui<-trunc((Yn/m)*SupportSize)

  if(Vi<= dgeom(Ui,0.1)){
    (SimulatedGeomSeq<-c(SimulatedGeomSeq,Ui))
  }
  k<-k+1
}
SimulatedGeomSeq

## [1] 13 1 9 5 3 1 0 18 14 10 6 0 29 8 5 12 1 8 4 3 0 17 13 10 6
## [26] 3 29 69 1 8 4 11 0 8 6 4 2 0 17 13 9 5 3 28 1 7 4 11 0 7
## [51] 6 3 2 16 12 9 5 2 27 0 7 3 10 7 5 3 1 20 16 12 8 2 0 27 0
## [76] 6 48 2 10 6 5 2 1 19 15 11 7 1 0 30 26 6 47 13 2 9 5 4 2 0
```

Dans la septième question, nous avons ajouté du code pour prolonger la génération de la même séquence. Tout d'abord, nous avons enregistré le dernier élément de la première séquence générée, X_n , en utilisant la syntaxe **$X_n \leftarrow \text{Seq1}[\text{length}(\text{Seq1})]$** . Ensuite, nous avons fourni une explication du fonctionnement de la boucle avec un commentaire.

Explication de la boucle

Nous avons utilisé une boucle "while" avec la condition suivante : "tant que la taille de la séquence "SimulatedGeomSeq" est inférieure à 100, nous continuons la génération de nombres".

Nous avons également souligné que X_n est le dernier élément de la séquence générée et que la fonction *gcl* utilise une variable globale appelée *x*.

Par conséquent, nous avons attribué à *x* la valeur de X_n pour permettre la continuité de la génération à partir de la dernière graine, $x \leftarrow X_n$.

Ensuite, nous avons expliqué que V_i est une variable utilisée pour générer un seul élément de la séquence, selon le même processus que précédemment. La différence est que cette fois-ci, nous ne générons qu'un seul élément à la fois, car nous sommes dans une boucle "while" et non dans une boucle "for".

Question 8

```
cat("Nombre d'itérations du GCL nécessaires pour obtenir une séquence de 100:",k*2)

## Nombre d'itérations du GCL nécessaires pour obtenir une séquence de 100: 2414
```

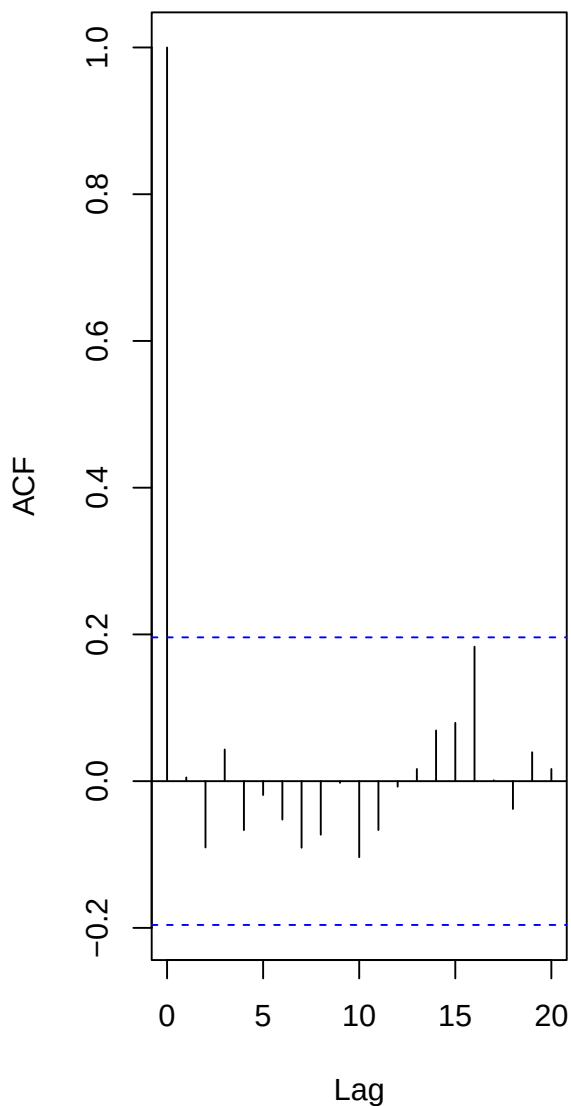
Nous avons initialisé k à 100 parce que nous avons déjà généré 100 nombres du GCL au préalable. Cette valeur est utilisée comme un compteur dans une boucle "while" qui vise à suivre le nombre de GCL utilisés pour générer une séquence de taille 100.

Ensuite, nous multiplions k par 2 car nous utilisons deux GCL de même taille pour chaque itération de la boucle. Cela permet de tenir compte du fait que deux GCL sont utilisés simultanément pour la génération de chaque nombre de la séquence.

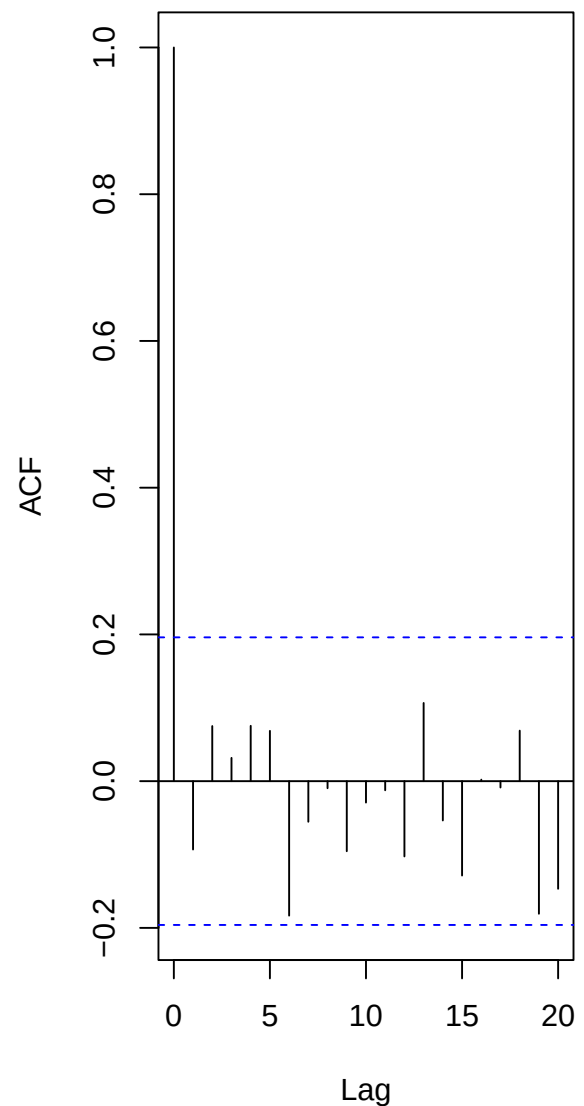
- `cat()` : est une fonction qui est utilisée pour afficher du texte ou des valeurs sur la sortie standard.

```
r<-rgeom(100,0.1)
par(mfrow=c(1,2))
acf(SimulatedGeomSeq)
acf(r)
```

Series SimulatedGeomSeq



Series r



Remarque

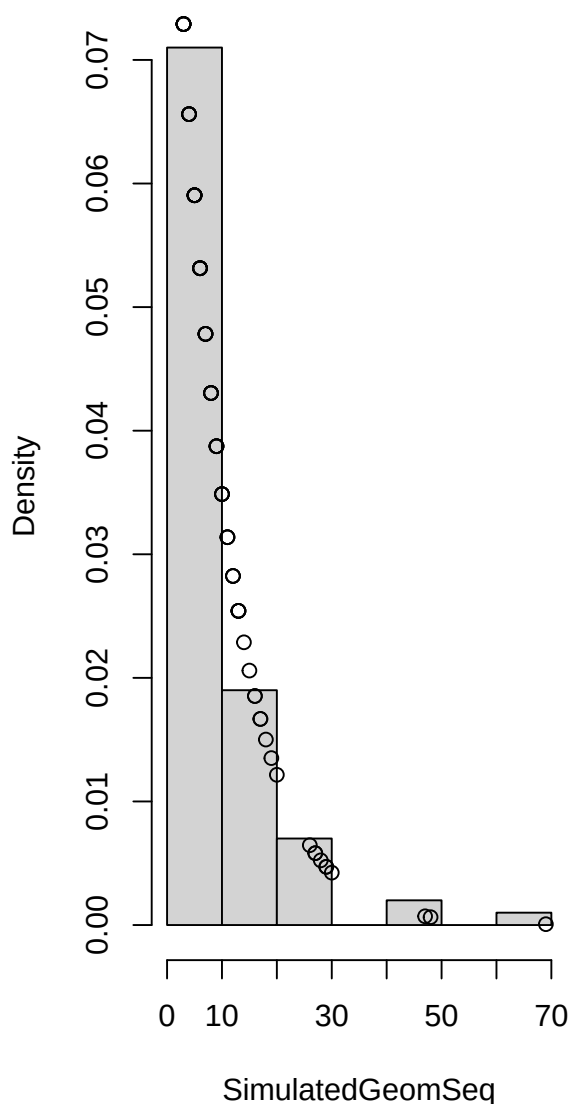
Il est remarquable de constater que la fonction `acf` appliquée à `SimulatedGeomSeq` présente une forte similarité avec celle appliquée à des séquences générées par `rgeom`.

Cette observation suggère que le comportement d'autocorrélation de `SimulatedGeomSeq` est étroitement lié à celui des séquences générées par `rgeom`.

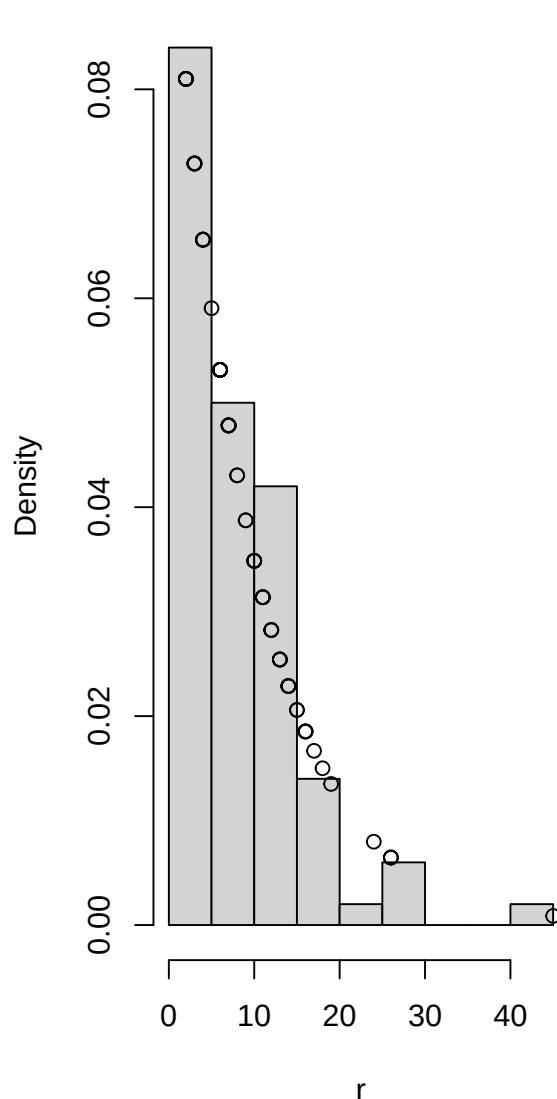
```
par(mfrow=c(1,2))
hist(SimulatedGeomSeq,freq = F)
points(SimulatedGeomSeq, dgeom(SimulatedGeomSeq,0.1))

hist(r,freq = F)
points(r, dgeom(r,0.1))
```

Histogram of SimulatedGeomSeq



Histogram of r



Remarque

Il est notable que l'histogramme de `SimulatedGeomSeq` présente une similitude frappante avec celui de `rgeom`.

Cette observation revêt une grande importance, car `rgeom` est largement reconnu comme le meilleur simulateur de séquences aléatoires suivant la loi géométrique. La similarité entre les deux histogrammes suggère que la méthode de rejet est également efficace.

Cette concordance entre les deux distributions indique que la méthode de rejet peut générer des séquences aléatoires de manière cohérente avec la loi géométrique, à l'instar de `rgeom`. Ainsi, cela renforce la crédibilité de la méthode de rejet comme approche valide pour simuler des nombres aléatoires selon la loi géométrique.

Conclusion

Bien que la méthode de rejet soit une méthode simple et très efficace pour générer des séquences aléatoires selon la loi géométrique, et qu'elle soit très similaire à `rgeom`, qui est reconnue comme la meilleure fonction pour simuler des séquences de nombres pseudo-aléatoires selon la loi géométrique, nous avons observé précédemment qu'il fallait 2414 réalisations de la fonction `gcl` (pseudo-uniforme) pour générer seulement 100 nombres de séquence, soit environ 24 fois sa taille.

Cela pose un problème computationnel dans le cas de séquences de grande taille, car le nombre élevé d'itérations nécessaires peut entraîner une utilisation excessive des ressources et des temps de calcul plus longs.

Code R

```
1 gcl<-function(a,b,m){
2   x<--(a*x+b)%%m
3
4   return(x)
5 }
6
7
8 m=1171*8*7*9
9 x<-789547
10 b=11*13*17*19
11 a=1171*4*7*3+1
12 (Seq1<-replicate(100,gcl(a,b,m)))
13
14 x<-25794
15 (Seq2<-replicate(100,gcl(a,b,m)))
16
17 (MaxGeom<-dgeom(0,0.1))
18
19 (SeqUniFerme<-Seq1/(m-1))
20 (SeqMaxPi<-SeqUniFerme*MaxGeom)
21
22 #Q4
23
24
25 (PremierElement<-MaxGeom/(m-1))
26 i<-0
27
28 while(dgeom(i,0.1)>=PremierElement){
29
30   i<-i+1
31 }
32
33 #Q5
34 (SupportSize<-i+1)
35 (SeqUniOuvert<-Seq2/(m))
36
37
38
```

```

39 (SimulatedGeomSupp<-trunc(SeqUniOuvert*SupportSize))
40 SimulatedGeomSeq<-c()
41 for (i in 1:100) {
42   if(SeqMaxPi[i]<= dgeom(SimulatedGeomSupp[i],0.1)){
43     SimulatedGeomSeq<-c(SimulatedGeomSeq,SimulatedGeomSupp[i])
44   }
45 }
46 }
47
48 X_n<-Seq1[length(Seq1)]
49 Y_n<-Seq2[length(Seq2)]
50 k<-100
51 while(length(SimulatedGeomSeq)<100){
52   x<-X_n
53   X_n<-gcl(a,b,m)
54
55   x<-Y_n
56   Y_n<-gcl(a,b,m)
57
58   Vi<-(X_n/(m-1))*MaxGeom
59   Ui<-trunc((Y_n/m)*SupportSize)
60
61   if(Vi<= dgeom(Ui,0.1)){
62     (SimulatedGeomSeq<-c(SimulatedGeomSeq,Ui))
63   }
64   k<-k+1
65 }
66 }
67
68 }
69
70 r<-rgeom(100,0.1)
71 cat("Nombre d'itérations du GCL nécessaires pour obtenir une séquence de 100 nombres
72   pseudo-aléatoires de la loi géométrique",k*2)
73 par(mfrow=c(1,2))
74 acf(SimulatedGeomSeq)
75 acf(r)
76 par(mfrow=c(1,2))
77 hist(SimulatedGeomSeq,freq = F)
78 points(SimulatedGeomSeq, dgeom(SimulatedGeomSeq,0.1))
79
80 hist(r,freq = F)
81 points(r, dgeom(r,0.1))

```